

Chapter 4

Step 2: Dataset Preparation

Once the problem is defined, the next task is to gather and preprocess relevant historical data used to train and test the predictive model. The quality and comprehensiveness of the dataset directly impacts the model's performance and ability to generalize to unseen data.

Data Collection

The first phase of dataset preparation involves data collection, which includes gathering historical price data, trading volumes, and other relevant market data for the assets in question. Additionally, macroeconomic indicators, company financial statements, and even alternative data sources such as sentiment analysis from news articles or social media can provide valuable insights. It is crucial to ensure that the data is sourced from reliable providers to maintain accuracy and integrity.

Exploratory Data Analysis

After the data is collected, we need to understand the nature of the dataset and its features. **Exploratory Data Analysis (EDA)** analyzes and visualizes the dataset to uncover patterns, detect anomalies, and confirm assumptions using summary statistics and charts. It also assists in determining subsequent actions for data preprocessing and model formulation.

In Python, EDA can be efficiently performed using libraries such as pandas (see [gnt.co/book-pandas](https://www.quantconnect.com/resources/guide-to-pandas)) for data manipulation and tools like Sweetviz (see [gnt.co/book-sweetviz](https://www.quantconnect.com/resources/guide-to-sweetviz)) for automated EDA reporting.

To install Sweetviz, run this command

```
pip install sweetviz
```

Let's illustrate how to use Sweetviz in an example of a dataset of Age, Income, Gender, and City.

```
import pandas as pd
import numpy as np
```

```
import sweetviz as sv

np.random.seed(42)

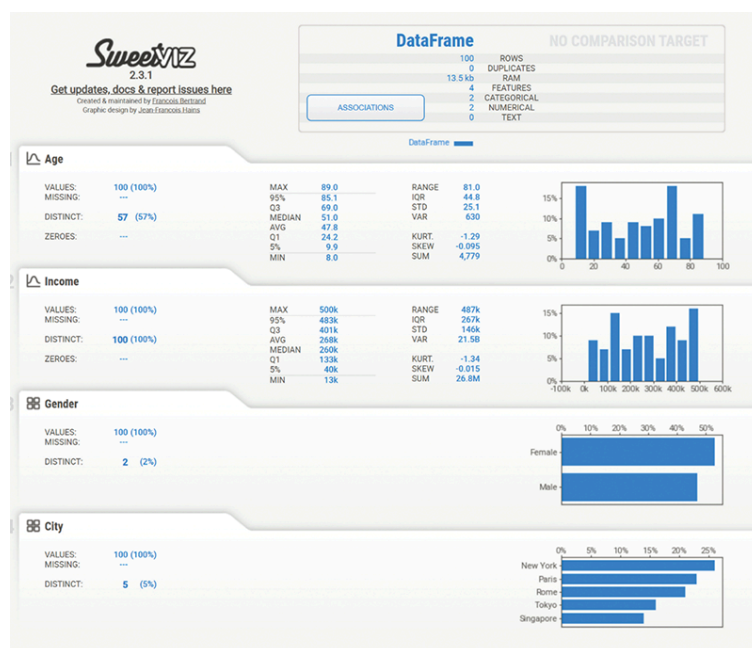
# Generate sample data
data = {
    'Age': np.random.randint(8, 90, size=100),
    'Income': np.random.randint(10000, 500000, size=100),
    'Gender': np.random.choice(['Male', 'Female'], size=100),
    'City': np.random.choice(['New York', 'Singapore', 'Paris', 'Rome', 'Tokyo'], size=100)
}

# Create a DataFrame
df = pd.DataFrame(data)

# Generate the general report
general_report = sv.analyze(df)
general_report.show_html('sweetviz_report.html')

# Generate report by gender
gender_report = sv.compare_intra(df, df['Gender'] == 'Male', ['Male', 'Female'])
gender_report.show_html('sweetviz_gender_comparison.html')
```

The code first generates an HTML file with a general interactive report on the dataset ([Figure 4.1](#)).



[Figure 4.1](#) Sweetviz dashboard with no comparison target.

It also generates an HTML report by gender using Sweetviz's method `compare_intra`, allowing us to compare two subsets of a dataset ([Figure 4.2](#)).



Figure 4.2 Sweetviz dashboard with gender as the comparison target.

Complete documentation is available at gnt.co/book-sweetviz-docs.

Data Preprocessing

Once we understand the nature of the input data, we move to the **preprocessing phase**, involving cleaning the dataset of data problems, such as missing values, the existence of outliers, removing duplicates, and correcting any errors or inconsistencies. These data problems can arise for various reasons, such as data entry errors, technical issues, market anomalies, or sudden unusual events. Furthermore, **normalization or standardization of features** may be necessary, mainly when dealing with variables that have different scales. For instance, transforming all features to a standard scale can improve the performance and convergence of specific machine-learning algorithms. Additionally, **transforming time series features to be stationary** is often required for enhanced accuracy and reliability of models in time series analysis to ensure that the statistical properties of the series remain constant over time. Finally, the **Engle-Granger test** is used to identify cointegrated pairs of time series, essential for developing mean-reverting pairs trading strategies.

Handling Missing Data

Dealing with missing data is essential to the data preprocessing pipeline, especially for financial AI applications. Incomplete datasets can cause skewed outcomes, lower model accuracy, and erroneous financial forecasts.

Identifying Missing Data

Before handling missing data, it is essential to identify where and how much data is missing. This can be achieved using pandas in Python:

```
import pandas as pd
import numpy as np
```

```

# Set random seed for reproducibility
np.random.seed(42)

# Generate synthetic financial data
n_samples = 1000
data = {
    'Date': pd.date_range(start='1/1/2023', periods=n_samples, freq='D'),
    'Open': np.random.uniform(100, 500, size=n_samples),
    'High': np.random.uniform(100, 500, size=n_samples),
    'Low': np.random.uniform(100, 500, size=n_samples),
    'Close': np.random.uniform(100, 500, size=n_samples),
    'Volume': np.random.randint(1000, 100000, size=n_samples)
}

# Convert Volume to float
data['Volume'] = data['Volume'].astype(float)

# Introduce some missing values for demonstration
data['Volume'][np.random.choice(n_samples, size=50, replace=False)] = np.nan
data['Close'][np.random.choice(n_samples, size=20, replace=False)] = np.nan

# Create a DataFrame
df = pd.DataFrame(data)

# Check for missing values
missing_data = df.isnull().sum()
print(missing_data)

```

Strategies for Handling Missing Data

1. **Removing Missing Values:** This approach is straightforward but can lead to significant data loss.

```

# Drop rows with missing values
df_cleaned = df.dropna()

# Drop columns with missing values
df_cleaned = df.dropna(axis=1)

```

2. **Imputation:** involves filling in the missing values using statistical methods or machine learning models. We need to remove the Date columns from the imputation in all the examples since imputation works only on numerical columns.

1. **Mean/Median/Mode Imputation:** Simple imputation can be done using the column's mean, median, or mode.

```

from sklearn.impute import SimpleImputer
# Using mean for imputation
imputer = SimpleImputer(strategy='mean')
df.iloc[:, 1:] = imputer.fit_transform(df.iloc[:, 1:])

```

2. **K-Nearest Neighbors (KNN) Imputation:** KNN imputation replaces missing values using the k-nearest neighbors' values.

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
df.iloc[:, 1:] = imputer.fit_transform(df.iloc[:, 1:])
```

3. **Multivariate Imputation by Chained Equations (MICE):** MICE performs multiple imputations by chaining together regression models.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
imputer = IterativeImputer()
df.iloc[:, 1:] = imputer.fit_transform(df.iloc[:, 1:])
```

Evaluating Imputation

After handling the missing data, we should evaluate the imputation's impact on the dataset with either visualizations or statistical measures to compare the original and imputed datasets.

Handling Outliers

Outliers in financial data can significantly skew the results of predictive models and lead to erroneous conclusions.

Identifying Outliers

The first step in handling outliers is to identify them. Standard techniques for identifying outliers include visualizations like box plots and scatterplots and statistical methods, such as the z-score and the interquartile range (IQR).

Using Box Plot to Identify Outliers

A box plot ([Figure 4.3](#)) provides a visual summary of the distribution of a dataset and highlights the presence of outliers.

```
import pandas as pd
import matplotlib.pyplot as plt
# Sample financial data
data = {'Price': [100, 95, 96, 101, 103, 98, 99, 500, 103, 110]}
df = pd.DataFrame(data)

# Box plot
plt.figure(figsize=(10, 6))
plt.boxplot(df['Price'])
plt.title('Box Plot for Price')
plt.ylabel('Price')
plt.show()
```

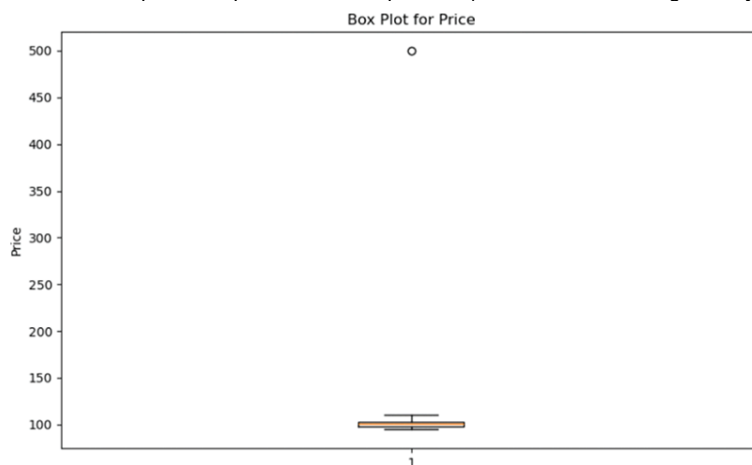


Figure 4.3 Box plot to identify outliers.

Figure 4.3 identifies an outlier of 500.

Using Z-score to Identify Outliers

The z-score indicates how many standard deviations an element is from the mean. Typically, a z-score greater than two or less than -2 , sometimes greater than three or less than -3 , is considered an outlier.

```
import pandas as pd
import numpy as np

# Sample financial data
data = {'Price': [100, 95, 96, 101, 103, 98, 99, 500, 103, 110]}
df = pd.DataFrame(data)

# Calculate Z-scores
df['Z_score'] = (df['Price'] - df['Price'].mean()) / df['Price'].std()
# Identify outliers
outliers = df[np.abs(df['Z_score']) > 2]
print("Outliers using Z-score method:")
print(outliers)
```

With the output

```
Outliers using the Z-score method:
   Price  Z_score
7    500  2.844444
```

Using Interquartile Range (IQR) to Identify Outliers

The IQR ranges between the first quartile (25th percentile) and the third quartile (75th percentile). Outliers are often defined as data points that lie beyond 1.5 times the IQR above the third quartile or below the first quartile.

```
import pandas as pd

# Sample financial data
data = {'Price': [100, 95, 96, 101, 103, 98, 99, 500, 103, 110]}
```

```
df = pd.DataFrame(data)

# Calculate IQR
Q1 = df['Price'].quantile(0.25)
Q3 = df['Price'].quantile(0.75)
IQR = Q3 - Q1

# Identify outliers
outliers = df[(df['Price'] < (Q1 - 1.5 * IQR)) | (df['Price'] > (Q3 + 1.5 * IQR))]
print("Outliers using IQR method:")
print(outliers)
```

With the output

```
Outliers using IQR method:
   Price
7     500
```

Handling Outliers

Once outliers are identified, there are several strategies to handle them. These include removing the outliers, transforming them, or capping/flooring their values.

1. **Removing Outliers:** removing outliers can be a straightforward approach, especially if the outliers are due to data entry errors.

```
import pandas as pd
import numpy as np

# Sample financial data
data = {'Price': [100, 95, 96, 101, 103, 98, 99, 500, 103, 110]}
df = pd.DataFrame(data)

# Calculate Z-scores
df['Z_score'] = (df['Price'] - df['Price'].mean()) / df['Price'].std()

# Remove outliers using the Z-score method
df_cleaned = df[np.abs(df['Z_score']) <= 2]
print("Data after removing outliers:")
print(df_cleaned)
```

With the output

```
Data after removing outliers:
   Price  Z_score
0     100 -0.320445
1      95 -0.360006
2      96 -0.352094
3     101 -0.312533
4     103 -0.296708
5      98 -0.336269
6      99 -0.328357
8     103 -0.296708
9     110 -0.241323
```

2. **Transforming Outliers:** transforming outliers can be helpful when the outliers are legitimate observations that represent essential market events. In this case, we log transform the entire dataset.

```
import pandas as pd
import numpy as np

# Sample financial data
data = {'Price': [100, 95, 96, 101, 103, 98, 99, 500, 103, 110]}
df = pd.DataFrame(data)

# Log transformation to reduce the impact of outliers
df['Price_log'] = np.log(df['Price'])
print("Data after log transformation:")
print(df)
```

With the output

```
Data after log transformation:
   Price  Price_log
0    100    4.605170
1     95    4.553877
2     96    4.564348
3    101    4.615121
4    103    4.634729
5     98    4.584967
6     99    4.595120
7    500    6.214608
8    103    4.634729
9    110    4.700480
```

3. **Capping/Flooring Outliers:** capping or flooring involves setting a threshold beyond which values are capped to reduce their influence.

```
import pandas as pd
import numpy as np

# Sample financial data
data = {'Price': [100, 95, 96, 101, 103, 98, 99, 500, 103, 110]}
df = pd.DataFrame(data)

# Calculate IQR
Q1 = df['Price'].quantile(0.25)
Q3 = df['Price'].quantile(0.75)
IQR = Q3 - Q1

# Capping outliers using IQR method
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df['Price_capped'] = np.where(df['Price'] > upper_bound, upper_bound,
                             np.where(df['Price'] < lower_bound, lower_bound, df['Price']))
print("Data after capping outliers:")
print(df)
```

With the output

Data after capping outliers:

	Price	Price_capped
0	100	100.000
1	95	95.000
2	96	96.000
3	101	101.000
4	103	103.000
5	98	98.000
6	99	99.000
7	500	110.125
8	103	103.000
9	110	110.000

Feature Engineering

Feature engineering is used to modify or create new features that can improve the performance of machine-learning models by transforming raw data into new values that better represent the underlying problem to the algorithms, enhancing their predictive power.

Techniques in feature engineering include normalization, standardization, encoding categorical variables, handling missing values, and creating interaction terms.

Let's illustrate with an example of creating two new moving averages features:

```
import pandas as pd
import numpy as np

# Set random seed for reproducibility
np.random.seed(42)

# Generate synthetic financial data
n_samples = 1000
data = {
    'Date': pd.date_range(start='1/1/2023', periods=n_samples, freq='D'),
    'Open': np.random.uniform(100, 500, size=n_samples),
    'High': np.random.uniform(100, 500, size=n_samples),
    'Low': np.random.uniform(100, 500, size=n_samples),
    'Close': np.random.uniform(100, 500, size=n_samples),
    'Volume': np.random.randint(1000, 100000, size=n_samples)
}

# Create a DataFrame
df = pd.DataFrame(data)

# Calculate moving averages
df['MA5'] = df['Close'].rolling(window=5).mean()    # 5-day moving average
df['MA10'] = df['Close'].rolling(window=10).mean() # 10-day moving average
```

Normalization and Standardization of Features

Normalization and standardization transform data to a standard scale, making it easier to compare and analyze. They are instrumental when working with features that have different units or scales, as they help im-

prove the performance and convergence of specific machine-learning algorithms.

Normalization

Normalization, known as min-max scaling, transforms the data to a specific range, usually between 0 and 1, by subtracting the minimum value of the feature, and then dividing the result by the range (maximum value minus minimum value).

The formula for normalization is

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}.$$

Normalization is beneficial for modeling algorithms with no data distribution assumption, such as neural networks or k-nearest neighbors.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Sample data
data = {
    'Feature1': [.4, .2, .1, .9, .6],
    'Feature2': [90, 101, 95, 94, 102],
    'Feature3': [9000, 10100, 9500, 9400, 10200]
}
df = pd.DataFrame(data)

# Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Fit and transform the data
normalized_data = scaler.fit_transform(df)

# Convert the normalized data back to a DataFrame
normalized_df = pd.DataFrame(normalized_data, columns=df.columns)

# Display the normalized DataFrame
print("Normalized DataFrame:")
print(normalized_df)
```

With the output

```
Normalized DataFrame:
   Feature1  Feature2  Feature3
0      0.375    0.000000    0.000000
1      0.125    0.916667    0.916667
2      0.000    0.416667    0.416667
3      1.000    0.333333    0.333333
4      0.625    1.000000    1.000000
```

Standardization

In finance, datasets often contain features with varying scales and units. For example, stock prices can range from a few dollars to several thou-

sand dollars, daily returns in percentages are very low, and trading quantities can be in the range of millions. Such disparities can negatively impact the performance of specific machine-learning algorithms sensitive to the scale of the input data, such as support vector machines, which would assign disproportionate influence on features with large scale.

The primary goal of standardization, also known as z-score normalization, is to ensure that each feature contributes equally to the model by scaling the features to have a mean of zero and a standard deviation of one.

The mathematical formula for standardization is

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- Z is the standardized value.
- X is the original value of the feature.
- μ is the mean of the feature, defined as

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i$$

N is the number of data points, and X_i is the feature value for the i th data point.

- σ is the standard deviation of the feature defined as

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2}$$

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Sample data
data = {
    'Feature1': [.4, .2, .1, .9, .6],
    'Feature2': [90, 101, 95, 94, 102],
    'Feature3': [9000, 10100, 9500, 9400, 10200]
}
df = pd.DataFrame(data)

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit and transform the data
standardized_data = scaler.fit_transform(df)

# Convert the standardized data back to a DataFrame
standardized_df = pd.DataFrame(standardized_data, columns=df.columns)

# Display the standardized DataFrame
print("Standardized DataFrame:")
print(standardized_df)
```

With the output

Standardized DataFrame:

	Feature1	Feature2	Feature3
0	-0.139347	-1.422574	-1.422574
1	-0.836080	1.022475	1.022475
2	-1.184446	-0.311188	-0.311188
3	1.602486	-0.533465	-0.533465
4	0.557386	1.244752	1.244752

Transforming Time Series Features to Stationary

One of the aims of time series analysis is to ensure that the features have consistent statistical properties throughout time. A time series is called stationary when its statistical properties, such as mean, variance, and autocorrelation remain constant over time. A time series model may have difficulty finding patterns and making reliable predictions when the time series is non-stationary.

Standard techniques to achieve stationarity include differencing, de-trending, and log transformation.

The Augmented Dickey-Fuller (ADF) test is a widely used method to test for stationarity. The ADF test helps determine if a time series has a unit root, indicating non-stationarity. If the test statistic is less than the critical value, the null hypothesis of a unit root is rejected, suggesting that the series is stationary.

Following is a Python example demonstrating the use of the ADF test with differencing:

```
import pandas as pd
import numpy as np
from statsmodels.tsa.stattools import adfuller
import matplotlib.pyplot as plt

# Generate sample non-stationary data
np.random.seed(42)
time_series = np.random.randn(100).cumsum()

# Perform ADF test
result = adfuller(time_series)
print('ADF Statistic:', result[0])
print('p-value:', result[1])
for key, value in result[4].items():
    print('Critical Values:')
    print(f'    {key}, {value}')

# Plot the time series
plt.figure(figsize=(10, 6))
plt.plot(time_series, label='Original Time Series')
plt.title('Non-Stationary Time Series')
plt.legend()
plt.show()

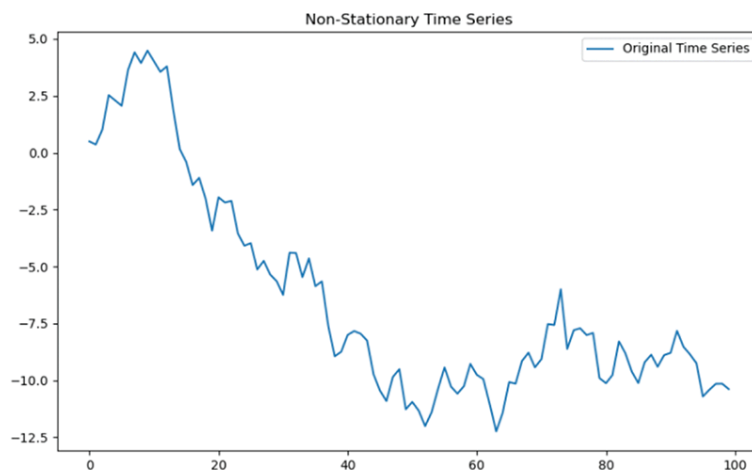
# Differencing to make the series stationary
diff_series = np.diff(time_series, n=1)
```

```
# Perform ADF test on differenced series
result_diff = adfuller(diff_series)
print('ADF Statistic (Differenced):', result_diff[0])
print('p-value (Differenced):', result_diff[1])
for key, value in result_diff[4].items():
    print('Critical Values (Differenced):')
    print(f'    {key}, {value}')

# Plot the differenced time series
plt.figure(figsize=(10, 6))
plt.plot(diff_series, label='Differenced Time Series')
plt.title('Stationary Time Series After Differencing')
plt.legend()
plt.show()
```

With the output ([Figures 4.4](#) and [4.5](#))

```
ADF Statistic: -1.3583317659818992
p-value: 0.6020814791099098
Critical Values:
    1%, -3.498198082189098
Critical Values:
    5%, -2.891208211860468
Critical Values:
    10%, -2.5825959973472097
ADF Statistic (Differenced): -10.008881137130237
p-value (Differenced): 1.800687720719554e-17
Critical Values (Differenced):
    1%, -3.4989097606014496
Critical Values (Differenced):
    5%, -2.891516256916761
Critical Values (Differenced):
    10%, -2.5827604414827157
```



[Figure 4.4](#) Example of a non-stationary time series.

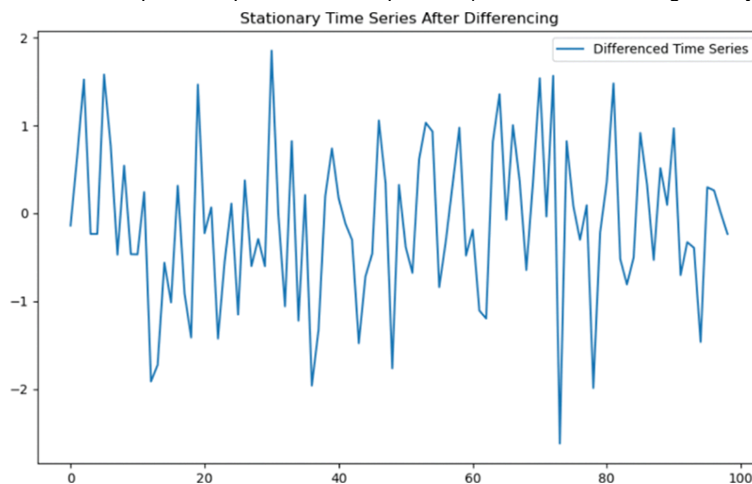


Figure 4.5 Example of a stationary time series that results from differentiating a non-stationary time series.

In finance, the recommended differentiating method is the fractional differentiation technique introduced by Lopez de Prado (2018, pp. 79–84), designed to avoid over-differentiating of the factors and to find a trade-off between stationarity and signal memory.

Let's redo the Python example demonstrating the use of the ADF test with fractional differentiation technique:

```
import pandas as pd
import numpy as np
from statsmodels.tsa.stattools import adfuller
import matplotlib.pyplot as plt

def get_weights_ffd(d, thres):
    """
    Computing the weights for differentiating the series with fixed window size

    Parameters:
        d (float): differentiating factor
        thres (float): threshold for cutting off weights

    Returns:
        w (np.ndarray): array containing weights
    """
    w, k = [1.0], 1
    while True:
        w_ = -w[-1] / k * (d - k + 1)
        if abs(w_) < thres:
            break
        w.append(w_)
        k += 1
    w = np.array(w[::-1]).reshape(-1, 1)
    return w

def frac_diff_ffd(series, d, thres=1e-5):
    """
    Fractional differentiation with constant width window
    Note 1: thres determines the cut-off weight for the window
    Note 2: d can be any positive fractional, not necessarily bounded [0,1]

    Parameters:
```

```

        series (pd.DataFrame): dataframe with time series
        d (float): differentiating factor
        thres (float): threshold for cutting off weights

    Returns:
        df (pd.DataFrame): dataframe with differentiated series
    ...

w = get_weights_ffd(d, thres)
width = len(w) - 1

df = {}
for name in series.columns:
    series_f = series[[name]].ffill().dropna()
    df_ = pd.Series(index=np.arange(series.shape[0]), dtype=object)
    for iloc1 in range(width, series_f.shape[0]):
        loc0, loc1 = series_f.index[iloc1 - width], series_f.index[iloc1]
        if not np.isfinite(series.loc[loc1, name]):
            continue # exclude NAs
        df_[loc1] = np.dot(w.T, series_f.loc[loc0:loc1])[0, 0]
    df[name] = df_.dropna().copy(deep=True)
df = pd.concat(df, axis=1)
return df

def ffd(process, thres=0.01):
    ...

    Finding the minimum differentiating factor that passes the ADF test

    Parameters:
        process (np.ndarray): array with random process values
        apply_constant_width (bool): flag that shows whether to use
            constant width window (if True) or increasing width window
            (if False)
        thres (float): threshold for cutting off weights
    ...

    for d in np.linspace(0, 1, 11):
        process_diff = frac_diff_ffd(pd.DataFrame(process), d, thres)
        test_results = adfuller(
            process_diff[process.name], maxlag=1, regression='c', autolag=None
        )
        if test_results[1] <= 0.05:
            break
    return process_diff[process.name]

# Generate sample non-stationary data
np.random.seed(42)
time_series = np.random.randn(100).cumsum()

# Perform ADF test
result = adfuller(time_series)
print('ADF Statistic:', result[0])
print('p-value:', result[1])
for key, value in result[4].items():
    print('Critical Values:')
    print(f'    {key}, {value}')

# Plot the time series
plt.figure(figsize=(10, 6))
plt.plot(time_series, label='Original Time Series')
plt.title('Non-Stationary Time Series')
plt.legend()
plt.show()

```

```

# Differencing to make the series stationary
diff_series = ffd(pd.Series(time_series, name='time_series'))

# Perform ADF test on differenced series
result_diff = adfuller(diff_series)
print('ADF Statistic (Differenced):', result_diff[0])
print('p-value (Differenced):', result_diff[1])
for key, value in result_diff[4].items():
    print('Critical Values (Differenced):')
    print(f'    {key}, {value}')

# Plot the differenced time series
plt.figure(figsize=(10, 6))
plt.plot(diff_series, label='Differenced Time Series')
plt.title('Stationary Time Series After Differencing')
plt.legend()
plt.show()

```

With the output

```

ADF Statistic: -1.3583317659818992
p-value: 0.6020814791099098
Critical Values:
    1%, -3.498198082189098
Critical Values:
    5%, -2.891208211860468
Critical Values:
    10%, -2.5825959973472097
ADF Statistic (Differenced): -3.614042461855659
p-value (Differenced): 0.0054981717565326035
Critical Values (Differenced):
    1%, -3.506057133647011
Critical Values (Differenced):
    5%, -2.8946066061911946
Critical Values (Differenced):
    10%, -2.5844100201994697

```

Given the ADF Statistic (Differenced): -3.614042461855659, let's consider typical critical values at the 1%, 5%, and 10% significance levels:

- 1%: -3.496
- 5%: -2.890
- 10%: -2.582

Since the value -3.614 is less than all critical values above, the differenced time series passes the ADF test at all 1%, 5%, and 10%, respectively, meaning it's stationary ([Figures 4.6](#) and [4.7](#)).

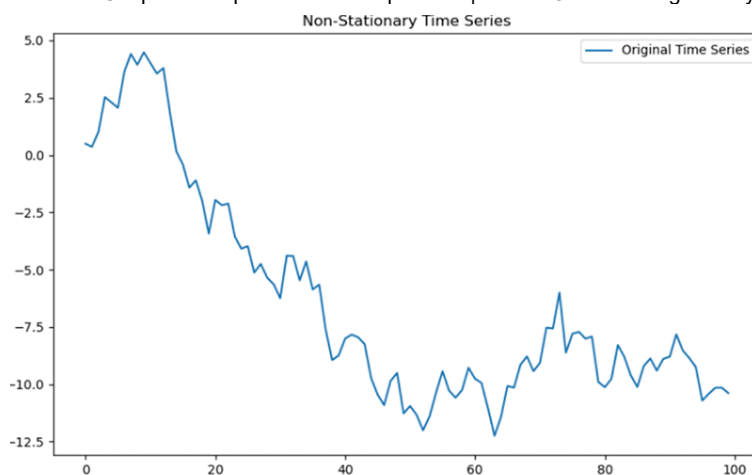


Figure 4.6 The original non-stationary time series before performing fractional differentiation.

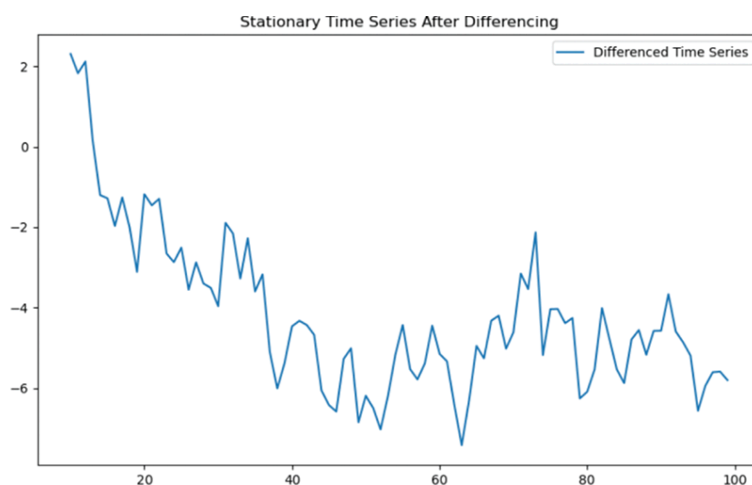


Figure 4.7 The stationary time series that results from performing fractional differentiation on the original non-stationary time series.

Identification of Cointegrated Time Series with Engle-Granger Test

Two or more non-stationary time series are cointegrated if the series have a stationary linear combination, indicating a stable long-term equilibrium relationship.

The Engle-Granger test is a widely used method to test for cointegration, allowing traders to identify pairs of assets that exhibit mean-reverting behavior, and then bet on the relative movement of two assets rather than their absolute prices.

Let us illustrate this with an example of two cointegrated non-stationary series (**Figure 4.8**).

```
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller, coint
import matplotlib.pyplot as plt

# Generate synthetic time series data
np.random.seed(42)
n = 100
time = np.arange(n)
```

```

# Simulate two non-stationary series (random walks)
asset1 = np.cumsum(np.random.randn(n)) + 41
asset2 = asset1 + np.random.randn(n)

# Create a DataFrame
data = pd.DataFrame({'asset1': asset1, 'asset2': asset2})

# Function to perform ADF test
def adf_test(series, name):
    result = adfuller(series)
    print(f'ADF Statistic for {name}: {result[0]}')
    print(f'p-value for {name}: {result[1]}')
    for key, value in result[4].items():
        print(f'Critical Value {key}: {value}')
    print('\n')

# Step 1: Test each series for stationarity
print("ADF Test for asset1:")
adf_test(data['asset1'], 'asset1')
print("ADF Test for asset2:")
adf_test(data['asset2'], 'asset2')

# Step 2: Perform the Engle-Granger cointegration test
score, pvalue, _ = coint(data['asset1'], data['asset2'])

print(f'Engle-Granger Cointegration Test score: {score}')
print(f'Engle-Granger Cointegration Test p-value: {pvalue}\n')

# Step 3: Visualize the series and their spread
data['spread'] = data['asset1'] - data['asset2']

plt.figure(figsize=(14, 7))
plt.subplot(2, 1, 1)
plt.plot(data['asset1'], label='Asset 1')
plt.plot(data['asset2'], label='Asset 2')
plt.legend()
plt.title('Non-Stationary Time Series')

plt.subplot(2, 1, 2)
plt.plot(data['spread'], label='Spread (Asset 1 - Asset 2)')
plt.legend()
plt.title('Spread (Should be Stationary)')
plt.tight_layout()
plt.show()

# Step 4: Test the spread for stationarity
print("ADF Test for spread:")
adf_test(data['spread'], 'spread')

```

With the output ([Figure 4.8](#))

And

```

ADF Test for asset1:
ADF Statistic for asset1: -1.3583317659819
p-value for asset1: 0.6020814791099095
Critical Value 1%: -3.498198082189098
Critical Value 5%: -2.891208211860468
Critical Value 10%: -2.5825959973472097

```

```

ADF Test for asset2:
ADF Statistic for asset2: -1.642378142665797
p-value for asset2: 0.4610184160167067
Critical Value 1%: -3.5019123847798657
Critical Value 5%: -2.892815255482889
Critical Value 10%: -2.583453861475781

Engle-Granger Cointegration Test score: -10.546923889518597
Engle-Granger Cointegration Test p-value: 1.0672395686754609e-17

ADF Test for spread:
ADF Statistic for spread: -10.875458876565842
p-value for spread: 1.3352313844030579e-19
Critical Value 1%: -3.498198082189098
Critical Value 5%: -2.891208211860468
Critical Value 10%: -2.5825959973472097

```

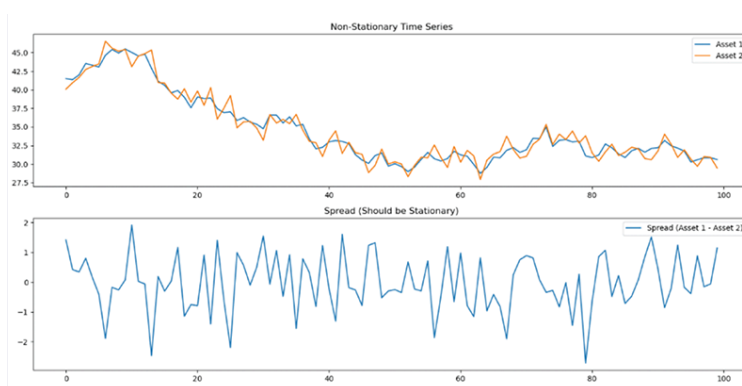


Figure 4.8 Two non-stationary assets that are cointegrated have a stationary spread.

To determine whether the time series is stationary based on the Augmented Dickey-Fuller (ADF) test statistic, we need to compare the ADF statistic to the critical values and check the p-value.

The critical values are the following:

- Critical Value 1%: -3.4996365338407074
- Critical Value 5%: -2.8918307730370025
- Critical Value 10%: -2.5829283377617176

Suppose the ADF statistic is less than the critical value. In that case, we reject the null hypothesis (the null hypothesis is that two series are not cointegrated) and conclude the time series are cointegrated.

For the asset1, the ADF statistic is -1.3583317659819 (> all critical values 1%, 5%, and 10%), meaning certainly non-stationary; for the asset2, the ADF statistic is -1.642378142665797 (> all critical values 1%, 5% and 10%) indicating again non-stationary, whereas the ADF statistic for the spread is -10.875458876565842 indicating the series are cointegrated. In sum, while the individual assets are non-stationary, their spread is stationary, and they are suitable for pairs trading.

Let us explain the concept of **p-value** on the p-value of the spread of 1.3352313844030579e-19. Assuming the null hypothesis is true, the p-value is the probability of obtaining the results at least as extreme as the observed results. A p-value lower than 0.05 indicates strong evidence against the null hypothesis, so we reject the null hypothesis. In this example, the data suggests that the series is cointegrated. If the p-value is more significant than 0.05, we fail to reject the null hypothesis; for cointegration, it would mean that the time series are not cointegrated.

Furthermore, the **Hurst coefficient**, a measure ranging from 0 to 1 of the nature of the long-term memory of a time series, can provide further insights into the nature of the cointegrated time series:

- $H < 0.5$: the time series exhibits mean-reverting behavior; that is, while time series can oscillate, they eventually return to their long-term average.
- $H = 0.5$: the time series exhibits no long-term memory and behaves like a random walk.
- $H > 0.5$: the time series follows a strong trend in the same direction.

A Hurst coefficient of the spread significantly less than 0.5 suggests a robust mean reversion ideal for pairs trading (a market-neutral strategy of two cointegrated securities, with a long position in the underperforming asset and a short position in the outperforming asset, expecting profit from their convergence to the equilibrium).

Let's illustrate the Hurst coefficient on a simple example of two cointegrated time series—notice that we have removed all non-positive values for calculating the Hurst exponent values—this is necessary since the calculation of the exponent depends on logarithms ([Figure 4.9](#)).

```
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller, coint
import matplotlib.pyplot as plt
from hurst import compute_Hc

# Generate synthetic cointegrated time series data
np.random.seed(42)
n = 1000 # Number of data points
time = np.arange(n)

# Generate a common stochastic trend
trend = np.cumsum(np.random.randn(n))

# Create two cointegrated series with added noise
asset1 = trend + 0.3 * np.random.randn(n)
asset2 = trend + 0.1 * np.random.randn(n)

# Create a DataFrame
data = pd.DataFrame({'asset1': asset1, 'asset2': asset2})

# Function to perform ADF test
def adf_test(series, name):
    result = adfuller(series)
    print(f'ADF Statistic for {name}: {result[0]}')
```

```

print(f'p-value for {name}: {result[1]}')
for key, value in result[4].items():
    print(f'Critical Value {key}: {value}')
print('\n')

# Test each series for stationarity
print("ADF Test for asset1:")
adf_test(data['asset1'], 'asset1')
print("ADF Test for asset2:")
adf_test(data['asset2'], 'asset2')

# Perform the Engle-Granger cointegration test
score, pvalue, _ = coint(data['asset1'], data['asset2'])

print(f'Engle-Granger Cointegration Test score: {score}')
print(f'Engle-Granger Cointegration Test p-value: {pvalue}\n')

# Calculate the spread
data['spread'] = data['asset1'] - data['asset2']

# Step 4: Test the spread for stationarity
print("ADF Test for spread:")
adf_test(data['spread'], 'spread')

# Remove zero and negative values in the spread for Hurst calculation
spread_non_zero = data['spread'][data['spread'] > 0]
if len(spread_non_zero) < len(data['spread']):
    print("Warning: Non-positive values in spread were removed for Hurst calculation.")

# Ensure spread_non_zero is not empty and does not contain invalid values
if len(spread_non_zero) > 0 and (spread_non_zero <= 0).sum() == 0:
    # Calculate the Hurst exponent of the spread
    H, c, data_hurst = compute_Hc(spread_non_zero, kind='price', simplified=True)
    print(f'Hurst Exponent for spread: {H}')

# Visualize the series and their spread
plt.figure(figsize=(14, 7))
plt.subplot(2, 1, 1)
plt.plot(data['asset1'], label='Asset 1')
plt.plot(data['asset2'], label='Asset 2')
plt.legend()
plt.title('Cointegrated Time Series')

plt.subplot(2, 1, 2)
plt.plot(data['spread'], label='Spread (Asset 1 - Asset 2)')
plt.legend()
plt.title('Spread (Should be Stationary)')
plt.tight_layout()
plt.show()
else:
    print("Error: Spread contains invalid values or is empty after removing non-positive values.")

```

With the outputs ([Figure 4.9](#))

And

```

ADF Test for asset1:
ADF Statistic for asset1: -0.9841906190427494
p-value for asset1: 0.7589866498633684

```

```
Critical Value 1%: -3.4369193380671
Critical Value 5%: -2.864440383452517
Critical Value 10%: -2.56831430323573
```

```
ADF Test for asset2:
ADF Statistic for asset2: -1.0039798833431093
p-value for asset2: 0.7518099637843659
Critical Value 1%: -3.4369127451400474
Critical Value 5%: -2.864437475834273
Critical Value 10%: -2.568312754566378
```

```
Engle-Granger Cointegration Test score: -31.604955223815125
Engle-Granger Cointegration Test p-value: 0.0
```

```
ADF Test for spread:
ADF Statistic for spread: -31.570817098575255
p-value for spread: 0.0
Critical Value 1%: -3.4369127451400474
Critical Value 5%: -2.864437475834273
Critical Value 10%: -2.568312754566378
```

```
Warning: Non-positive values in spread were removed for Hurst calculation.
Hurst Exponent for spread: 0.3099860341085018
```

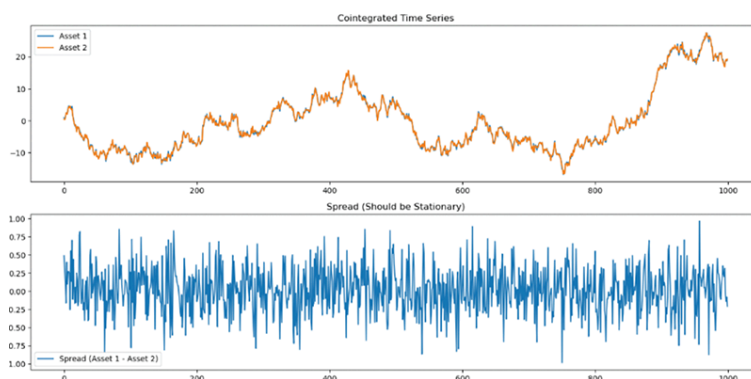


Figure 4.9 Two cointegrated assets and their stationary spread.

The Hurst exponent of 0.3 indicates that the assets would be ideal for pairs trading.

Feature Selection

The main goal of feature selection is to identify the most relevant features that contribute to the predictive power of a model by eliminating irrelevant features, which also helps reduce overfitting and improve the model's interpretability.

Let's explore four critical methods of feature selection: correlation analysis, feature importance analysis, auto-identification of features, and dimensionality reduction with principal component analysis (PCA).

Correlation Analysis

Correlation analysis helps identify features that are highly correlated with the target variable and features that are highly correlated with each other.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Generate random sample data with correlations
np.random.seed(42)
size = 100
feature1 = np.random.randn(size)
feature2 = feature1 + np.random.randn(size) * 0.1 # Highly correlated with feature1
feature3 = np.random.randn(size)
target = feature1 * 0.5 + np.random.randn(size) * 0.1 # Highly correlated with target

data = {
    'Feature1': feature1,
    'Feature2': feature2,
    'Feature3': feature3,
    'Target': target
}
df = pd.DataFrame(data)

# Calculate correlation matrix
corr_matrix = df.corr()

# Display correlation matrix
print(corr_matrix)

# Plot heatmap
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

With the output ([Figure 4.10](#))

	Feature1	Feature2	Feature3	Target
Feature1	1.000000	0.994477	0.190840	0.980875
Feature2	0.994477	1.000000	0.188663	0.974603
Feature3	0.190840	0.188663	1.000000	0.193556
Target	0.980875	0.974603	0.193556	1.000000

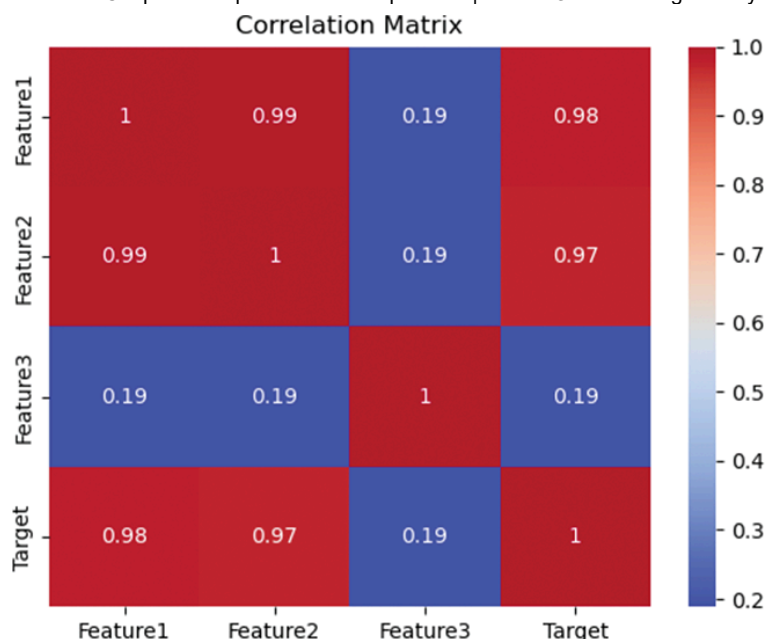


Figure 4.10 Correlation matrix of the target and factors.

Feature Importance Analysis

Tree-based machine-learning models, such as random forests, are frequently used for feature importance analysis as they naturally determine the importance of each feature in predicting the target variable.

To avoid selecting highly correlated features, we will first remove highly correlated features (correlation coefficient > 0.9) before running the feature importance analysis.

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt

# Generate random sample data with correlations
np.random.seed(42)
size = 100
feature1 = np.random.randn(size)
feature2 = feature1 + np.random.randn(size) * 0.1 # Highly correlated with feature1
feature3 = np.random.randn(size)
target = feature1 * 0.5 + np.random.randn(size) * 0.1 # Highly correlated with target

data = {
    'Feature1': feature1,
    'Feature2': feature2,
    'Feature3': feature3,
    'Target': target
}
df = pd.DataFrame(data)
X = df.drop('Target', axis=1)
y = df['Target']

# Remove highly correlated features
corr_matrix = X.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.9)]
```



```
X_reduced = X.drop(columns=to_drop)

# Train a Random Forest model
model = RandomForestRegressor()
model.fit(X_reduced, y)

# Get feature importance
importances = model.feature_importances_
feature_names = X_reduced.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})

# Display feature importance
print(importance_df)

# Plot feature importance
importance_df.sort_values(by='Importance', ascending=False).plot(kind='bar', x='Feature', y='Importance')
plt.title('Feature Importance')
plt.show()
```

With the output ([Figure 4.11](#))

	Feature	Importance
0	Feature1	0.984778
1	Feature3	0.015222

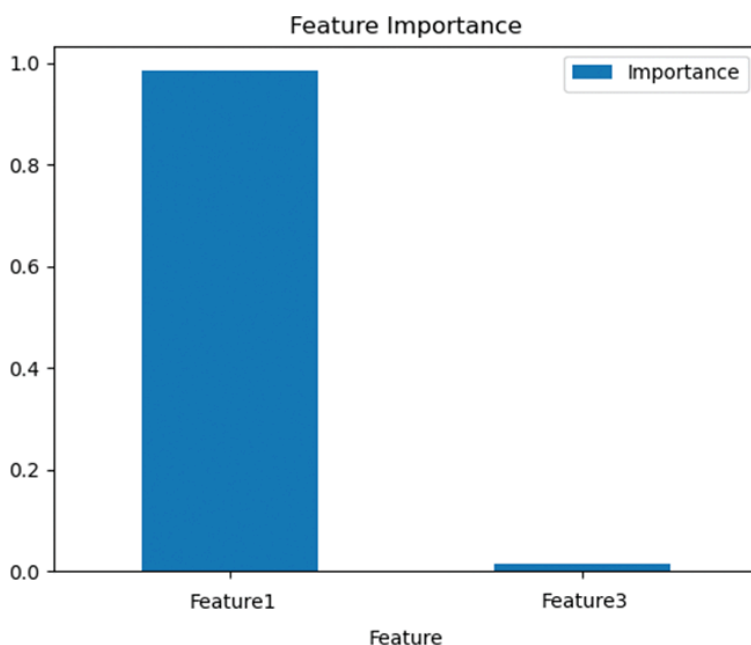


Figure 4.11 Feature importance plot of features 1 and 3. Feature 3 could be discarded to simplify the model and reduce the dataset dimensions.

Auto-identification of Features

Auto-identification of features involves using algorithms and statistical methods to automatically select the most relevant features for a given predictive model. Techniques such as recursive feature elimination (RFE) and sklearn's `SelectFromModel` are commonly used.

When features are highly correlated, RFE, which iteratively removes the least important features and refits the model, might select multiple corre-

lated features because they all seem essential in isolation. Therefore, we remove such features.

```
import pandas as pd
import numpy as np
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt

# Generate random sample data with correlations
np.random.seed(42)
size = 100
feature1 = np.random.randn(size)
feature2 = feature1 + np.random.randn(size) * 0.1 # Highly correlated with feature1
feature3 = np.random.randn(size)
target = feature1 * 0.5 + np.random.randn(size) * 0.1 # Highly correlated with targetfeature1

data = {
    'Feature1': feature1,
    'Feature2': feature2,
    'Feature3': feature3,
    'Target': target
}
df = pd.DataFrame(data)
X = df.drop('Target', axis=1)
y = df['Target']

# Remove highly correlated features
corr_matrix = X.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.9)]
X_reduced = X.drop(columns=to_drop)

# Apply Recursive Feature Elimination
model = RandomForestRegressor()
rfe = RFE(estimator=model, n_features_to_select=2)
rfe.fit(X_reduced, y)

# Get selected features
selected_features = X_reduced.columns[rfe.support_]
print(f'Selected Features: {selected_features}')

# Plot selected features
importance_df = pd.DataFrame({'Feature': X_reduced.columns, 'Importance': rfe.support_.astype(int)})
importance_df.plot(kind='bar', x='Feature', y='Importance')
plt.title('Auto-Identification of Features')

# Adjust the bottom margin to add more space for x-axis labels
plt.subplots_adjust(bottom=0.2)

plt.show()
```

With the output ([Figure 4.12](#))

```
Selected Features: Index(['Feature1', 'Feature3'], dtype='object')
```

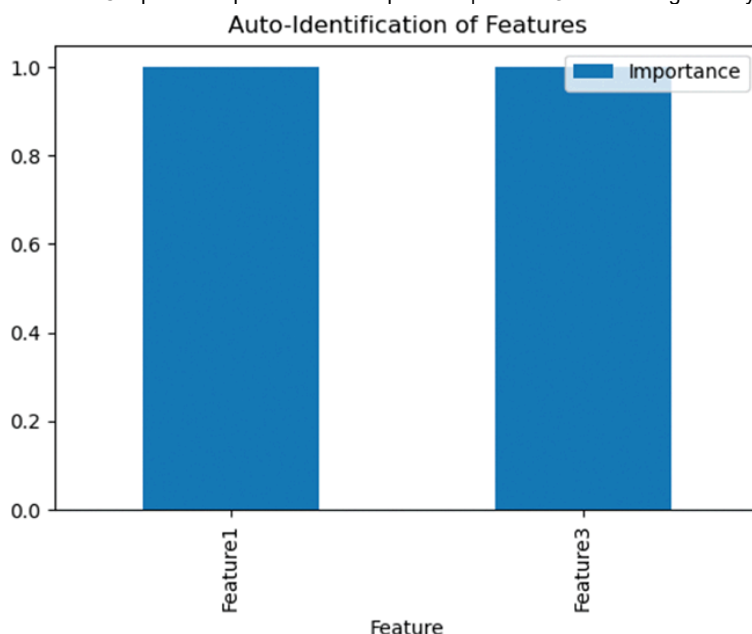


Figure 4.12 Importance plot of features 1 and 3 that were selected by recursive feature elimination.

Dimensionality Reduction/Principal Component Analysis

Dimensionality refers to the number of features in a dataset. The higher the number of features there is in a dataset, the more computationally intensive the models are, the more memory is required, the more difficult is to find any sensible patterns within the dataset, and the more likely the model is to overfit.

Principal component analysis (PCA) is a statistical technique that simplifies the analysis of complex data sets with many features by identifying a new smaller set of features that capture most of the underlying data's variation.

The newly identified features are called the **principal components**. They are a linear combination of the original features and are uncorrelated. They are ordered in descending order, so that the first principal component captures the most variance.

Standardization is crucial for PCA to ensure that each feature contributes equally to the principal components.

In finance, PCA is used for risk management, portfolio optimization, and reducing the complexity of large datasets while retaining important information.

These are the general steps to implement the PCA:

1. Calculate the covariance matrix by calculating the covariance between each pair of features.
2. Calculate the eigenvectors (they represent the direction of principal components) and eigenvalues (they represent the amount of variance explained by each principal component) of the covariance matrix.
3. Determine the number of principal components of interest; this is usually done by sorting the eigenvalues by the **explained variance ratio**

(the proportion of the dataset's total variance captured by each principal component) in descending order and choosing the top K eigenvectors, which in aggregate explain the desired percentage of the total variance, e.g., 90%.

4. Create a feature matrix consisting of the selected top K eigenvectors as columns.
5. Project the original standardized data onto the selected principal components by multiplying the original data with the feature vector matrix.

In Python, the sklearn library provides PCA implementation.

```
import pandas as pd
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Generate random sample data with correlations
np.random.seed(42)
size = 100
feature1 = np.random.randn(size)
feature2 = feature1 + np.random.randn(size) * 0.1 # Highly correlated with feature1
feature3 = np.random.randn(size)
target = feature1 * 0.5 + np.random.randn(size) * 0.1 # Highly correlated with feature1

data = {
    'Feature1': feature1,
    'Feature2': feature2,
    'Feature3': feature3,
    'Target': target
}
df = pd.DataFrame(data)
X = df.drop('Target', axis=1)

# Standardize the data
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(X_standardized)

# Create a DataFrame with the principal components
pca_df = pd.DataFrame(data=principal_components, columns=['Principal Component 1', 'Principal Component 2'])

# Display the explained variance ratio
print(f'Explained variance ratio: {pca.explained_variance_ratio_}')

# Plot the principal components
plt.scatter(pca_df['Principal Component 1'], pca_df['Principal Component 2'])
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Principal Component Analysis')
plt.show()
```

With the output ([Figure 4.13](#))

Explained variance ratio: [0.68742233 0.3107374]

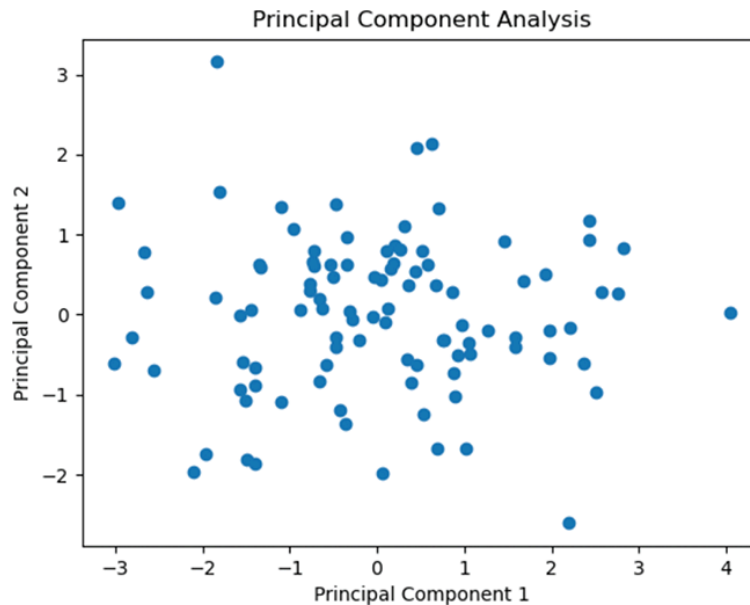


Figure 4.13 PCA transformed the feature matrix from three dimensions to two new dimensions that maximize the variance in the dataset.

In the scatterplot, each point represents a sample from the original dataset projected onto the plane defined by the first two components. The spread of the points along the axes of the principal components indicates how much variance is captured by each component.

Splitting of Dataset into Training, Testing, and Possibly Validation Sets

In general, it is recommended to separate your dataset into training, testing, and possibly validation sets when developing and evaluating machine learning models:

1. **Training Set:** this subset of the data is used to train your model. The model learns the patterns and relationships from this data.
2. **Testing Set:** after training, the model is evaluated on this set to assess its performance and generalization to unseen data. It helps identify any overfitting or underfitting issues. Overfitting is a modeling error where a model becomes too complex because it memorizes the training data, including its data noise and outliers, too well, resulting in poor generalization of new, unseen data.
On the other hand, underfitting is a modeling error where a model is too simple to capture the underlying patterns in the data, resulting in poor performance on the training and new, unseen data.
3. **Validation Set (Optional):** this set is used during training to tune hyperparameters. Hyperparameters are machine-learning settings that are preset before the learning starts controlling the learning process itself. Unlike model parameters learned from the training dataset, the hyperparameters are predefined—the Validation Sets help prevent overfitting by providing an additional check during the training phase.

How to Split Your Data

1. Training and Testing Split

1. A common approach is to split the data into 70–80% for training, and 20–30% for testing.
2. Use functions like `train_test_split` from libraries such as `sklearn` to perform this split.

2. Training, Validation, and Testing Split

1. If you use a validation set, a typical approach is to split the data into 60% for training, 20% for validation, and 20% for testing.

Here's an example of how you might split your data using `sklearn`:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Generate random sample data
np.random.seed(42)
size = 100
feature1 = np.random.randn(size)
feature2 = feature1 + np.random.randn(size) * 0.1 # Highly correlated with feature1
feature3 = np.random.randn(size)
target = feature1 * 0.5 + np.random.randn(size) * 0.1 # Highly correlated with feature1

data = {
    'Feature1': feature1,
    'Feature2': feature2,
    'Feature3': feature3,
    'Target': target
}
df = pd.DataFrame(data)

# Split the data into training and testing sets
X = df.drop('Target', axis=1)
y = df['Target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Print the shapes of the training and testing sets
print(f'Training features shape: {X_train.shape}')
print(f'Training target shape: {y_train.shape}')
print(f'Testing features shape: {X_test.shape}')
print(f'Testing target shape: {y_test.shape}')
```

With the output

```
Training features shape: (80, 3)
Training target shape: (80,)
Testing features shape: (20, 3)
Testing target shape: (20,)
```

The `sklearn`'s `train_test_split` function

```
sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None)
```

has these parameters:

1. **arrays**
 1. The input data (features and labels) you want to split. Typically, these are your feature matrix `X` and target vector `y`.
2. **test_size**
 1. Determines the proportion of the dataset to include in the test split. It can be a float (between 0.0 and 1.0), an integer (number of test samples), or `None`. If `None`, it will be set to the complement of the `train_size`.
3. **train_size**
 1. Determines the proportion of the dataset to include in the train split. It can be a float (between 0.0 and 1.0), an integer (number of train samples), or `None`. If `None`, it will be set to the complement of the `test_size`.
4. **random_state**
 1. Controls the shuffling applied to the data before applying the split. Pass an integer for reproducible output across multiple function calls.
5. **shuffle**
 1. Whether or not to shuffle the data before splitting. The default is `True`.
6. **stratify**
 1. If this is not `None`, data is divided proportionally, using this as the class label. This helps with classification problems to keep the balance of the target variable in the train and test sets.

If your dataset is small or imbalanced (in a classification problem, one class of dataset significantly outnumbered the others) or you are looking for a more robust assessment of the model robustness, you should use **cross-validation**. Cross-validation is a statistical method based upon splitting the dataset into multiple subsets (“folds”), and then iteratively training and testing the model on different combinations of these folds.

The most common form of cross-validation is **k-fold cross-validation**, where the dataset is split into `k`-equally sized folds with the model trained on `k-1` folds and tested on the remaining fold. This process is repeated `k`-times with each fold serving once the role of the test set. The model performance metric is then defined as the average of the performance metrics for each of the `k` runs.

The clear benefit of cross-validation is the more reliable estimation of the model performance than with a single test set. However, cross-validation is more computationally intensive than using a single test set.

Let’s compare the performance of the test-train approach with the `k`-fold cross-validation in an example of classifying the Wine dataset from the UCI Machine Learning Repository. The dataset is composed of the chemical analysis of three different types of wine grapes grown in Italy.

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```

# Load dataset
data = load_wine()
X, y = data.data, data.target

# Initialize the model
model = RandomForestClassifier(random_state=42)

# Perform 5-fold cross-validation
cv_scores = cross_val_score(model, X, y, cv=5)
print("Cross-validation scores:", cv_scores)
print("Mean cross-validation score:", cv_scores.mean())

# Perform a typical train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Test the model
y_pred = model.predict(X_test)
test_accuracy = accuracy_score(y_test, y_pred)
print("Train-test split accuracy:", test_accuracy)

# Compare the results
print("\nComparison of Results:")
print(f"Mean cross-validation score: {cv_scores.mean():.4f}")
print(f"Train-test split accuracy: {test_accuracy:.4f}")

```

With the output

```

Cross-validation scores: [0.97222222 0.94444444 0.97222222 0.97142857 1.]
Mean cross-validation score: 0.9720634920634922
Train-test split accuracy: 1.0

Comparison of Results:
Mean cross-validation score: 0.9721
Train-test split accuracy: 1.0000

```

The results indicate that the model performs well on the given dataset; however, the perfect score on the train-test split indicates potential overfitting.